DBMS- ASSIGNMENT 2 - CS 2604

Q1.

Consider the following definition of a B-tree :

A B-tree of order m is a self-balancing multi-way search tree with the following properties :-

a. Each node can have at most m children and at most m-1 key
b. The keys within each node are stored in order and are used to guide the search among its children.
c. The root is either a leaf or has at least two children
d. Every non-root internal node has at least $\lceil \frac{m}{2} \rceil$ children
e. All leaf nodes are at the same level; hence, the tree is height-balanced.

· Suggest an efficient algorithm to merge

A possible algorithm is to merge the two B tree can be -

1. Merging the Roots.

Extract the roots of T1 and T2. Combine their keys into one 'super-Root' list which is sorted and remove duplicates.

2. Creating Internal Bucket.

The keys from above step act as boundaries, forming K+1 child intervals (eg- <4, 4-10, 11-16, >16)

Form an empty 'super-leaf' bucket for every interval.

3. Distributing the leaves

Go through the leaves of both original trees. Put each of their keys into corresponding bucket according to the Super-Root boundaries.

4. Re-structuring

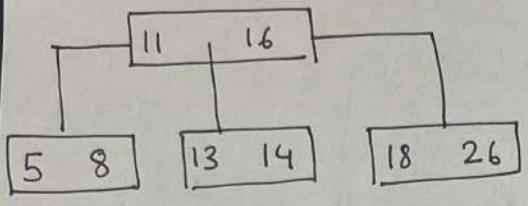
Go through the super-leaf bucket. For each bucket with $\geq m$ keys (ie, ≥ 4 keys), perform a standard B-tree split. It involves cutting the bucket at the median, producing two proper leaf nodes and puting the median into the Super-Root ~~now~~. If the Super-Root now contains more than $m-1$ keys (taking medians from lower level as its), split it. It means promoting its median key and forming a brand new root of the tree.

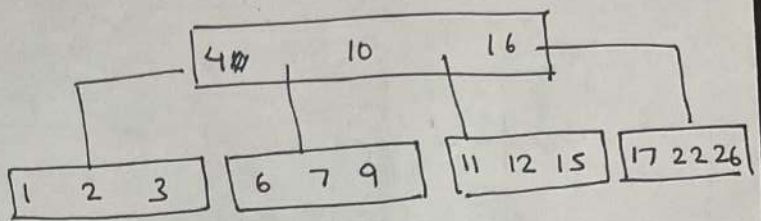
If

5. Producing NULL leave in any case

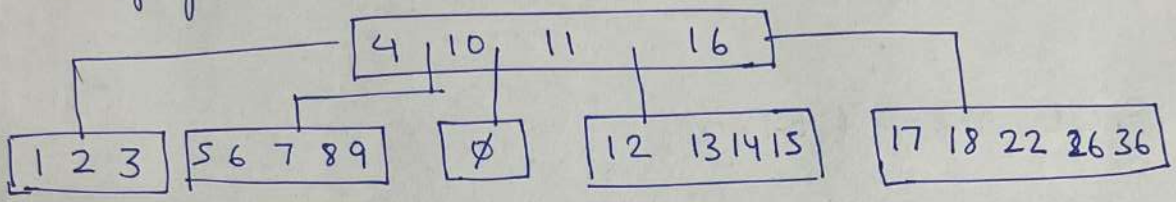
If we produce NULL in leave given any merge we need to follow standard B tree merging protocol to fix the null in the leave.

Execution Trace:-

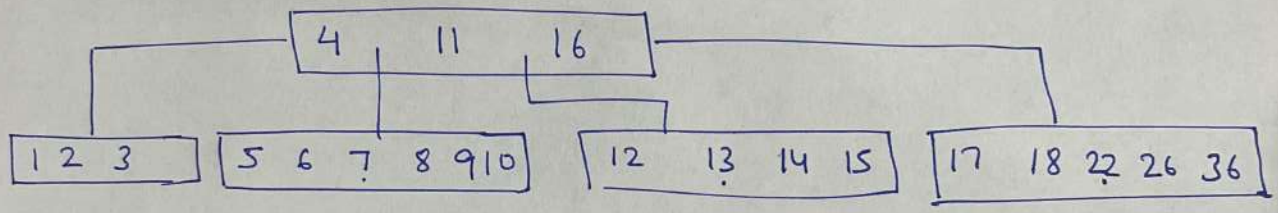




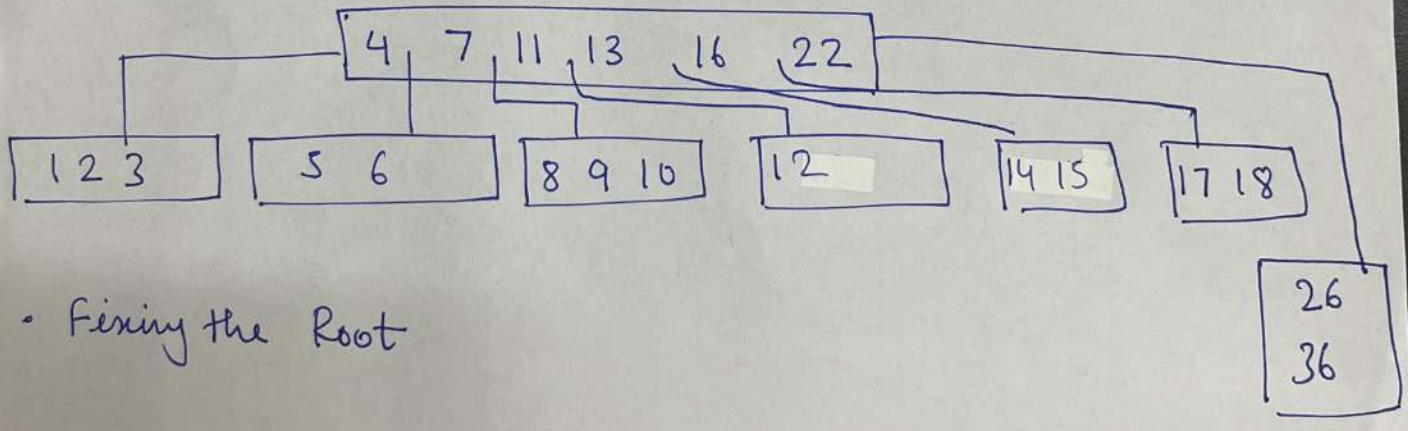
- Merging the Root



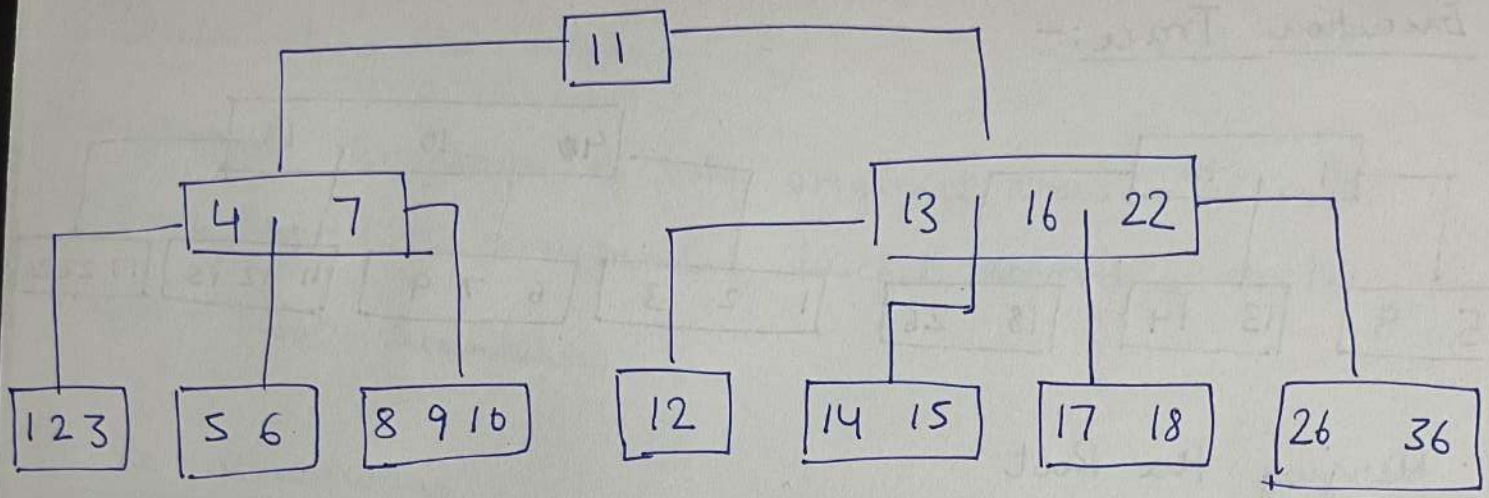
- Fixing the NULL



- Fixing the Imbalanced leaves.



- Fixing the Root



* Final merged B-Tree.

Q4 T2.

2. Given a relations R(A,B,C) that has all the attributes as atomic and have a functional dependency A → BC, Prove or disprove that R is in 5NF but not in 6NF.

Given :- R(ABC) with atomic entries and FD :- A → BC

To prove :- R is in 5NF & not in 6NF.

Proof :-

As we have, FD : A → BC.

we know, CK of R is A [∵ $A^+ = ABC$].

* Checking R is in 5NF or not.

we know, a relation R is in 5NF when it is in 4NF and every non-trivial Join dependency is implied by its candidate keys.

Here, we know, R(ABC) is in 4NF,

Since it do not any non-trivial MVD

Checking for Join dependencies,

As, A → BC can be decomposed into A → B and A → C.

we can decompose R into R1(AB) and R2(AC).

So, it corresponds to the JD (AB, AC). and this Join dependecy JD is implied by our candidate key, A. as A ⊆ AB and A ⊆ AC.

Hence R(ABC) satisfies our 5NF definition, hence in 5NF.

. Checking for 6NF?

We know, a relation R is in 6NF when it is in 5NF and it cannot be further decomposed losslessly into smaller tables.

We have R(ABC) FD = {A → BC}

So non key attributes are B, C we can decompose, R by taking primary key and each non key attribute once.

⇒ R
- → R1(AB)
- → R2(AC)

But there exists a lossless decomposion of R ⇒ R is not the smallest relation, so,

R is not in 6NF.

So our relation R(ABC) is proved to be in 5NF, but not 6NF